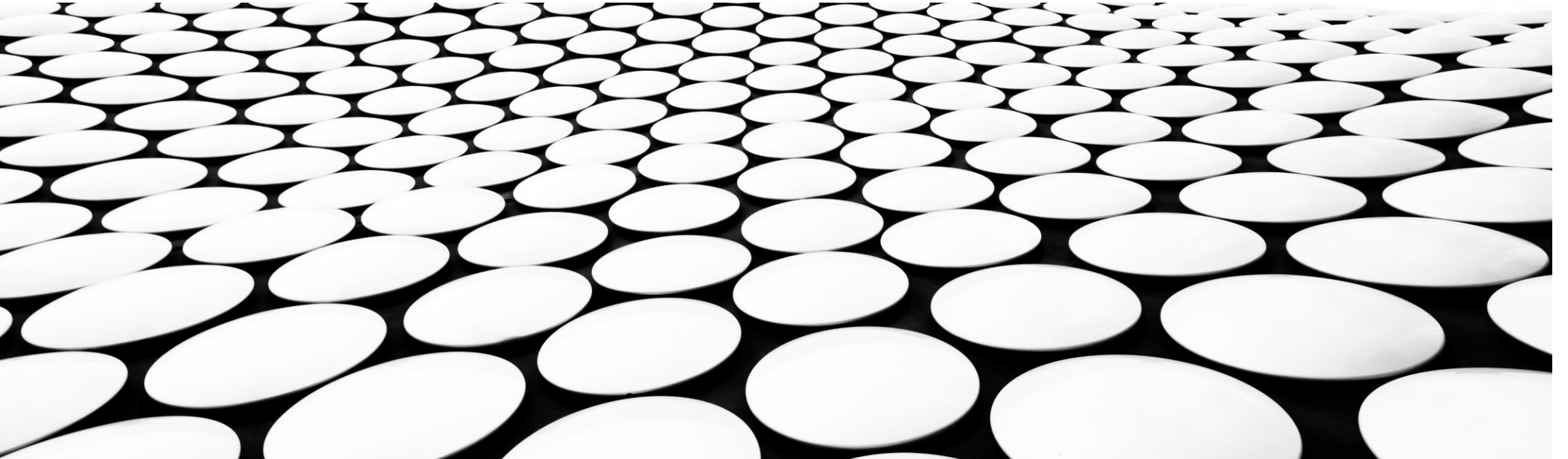

QUICK SORT ALGORITHM

MODULE III



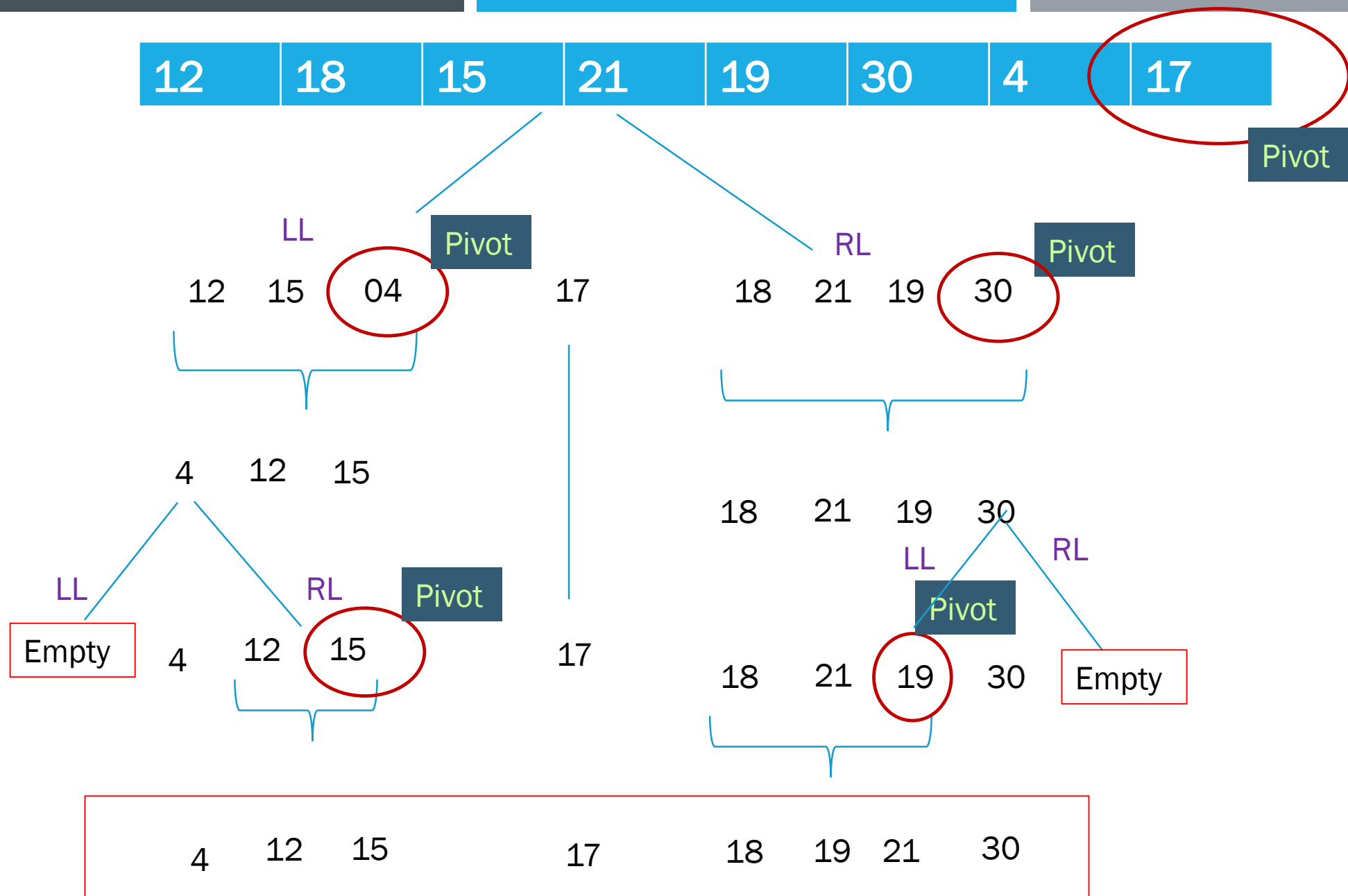
PRINCIPLE OF QUICK SORT ALGORITHM

- A divide and Conquer rule:
- **Divide:** partition array into 2 subarrays such that elements in the lower part $\leq$ elements in the higher part.
- **Conquer:** recursively sort the 2 subarrays.
- **Combine:** trivial since sorting is done in place.

TIME COMPLEXITY

- $O(n \log n)$ is the time complexity for quick sort and merge sort algorithm.
- Average case time complexity for quick sort is $O(n \log n)$
- Worst case time complexity for quick sort is $O(n^2)$

WORKING OF QUICK SORT



QUICKSORT WORKING CNTD..

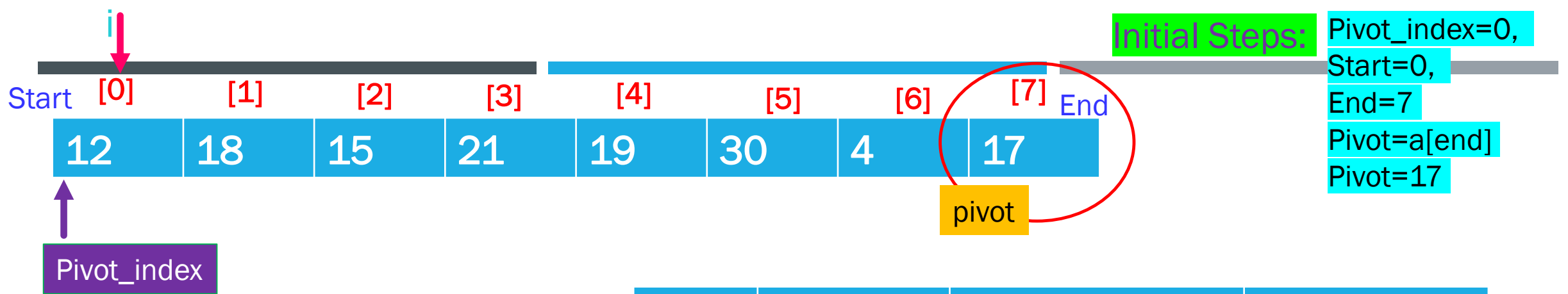
- At each step we choose a pivot element. Right most element or randomized element
- **Partitioning:** We bring all the elements smaller than pivot to the left side of pivot and larger than pivot to the right side such that pivot is at its correct position.
- We can solve recursively right side and left sides. Quick sort is also a recursive sorting algorithm. $O(n^2)$
- Quicksort(A,start,end); $T(n) = 2 * T(n/2) + c * n$
- Pivot_index Partition(A,start,end); $- c * n$
- Quicksort(A,start,pivot_index-1); $T(n/2)$
- Quicksort(A,pivot_index+1,end); $T(n/2)$
- If start $\geq$ end return c'

QUICK SORT ALGORITHM

- Quicksort(A,start,end)
- Begin
- If $\text{start} \geq \text{end}$ return.
- Else
- $\text{pivot_index} = \text{Partition}(A, \text{start}, \text{end});$
- Quicksort(A,start,pivot_index-1);
- Quicksort(A,pivot_index+1,end)
- End.

PARTITION ALGORITHM

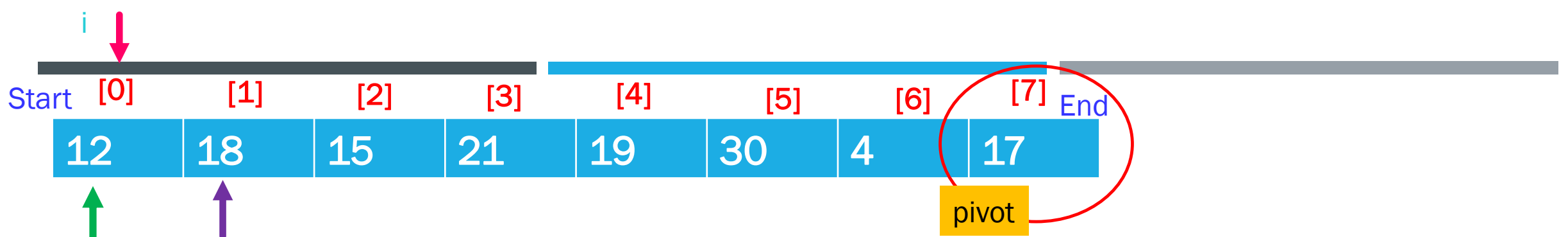
- Partition(A,start,end)
- Begin
- For i=start to end-1
- { if a[i] <= pivot
- { swap(a[i],a[p_index])
- ++pivot_index;
- }
- }
- Swap(a[end],a[pivot_index])
- Return pivot_index;
- End.



i=start	A[i] <=pivot	Swap(a[i],a[p_index])	++pivot_index
0			

- For i=start to end-1
- { if a[i] <= pivot
- { swap(a[i],a[p_index])
- ++pivot_index;
- } }
- Swap(a[end],a[pivot_index])
- Return pivot_index;

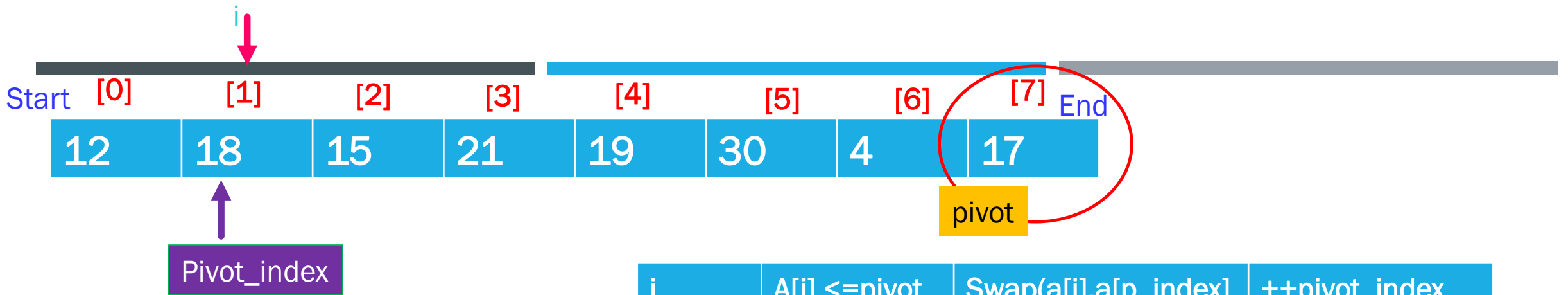
PARTITION ALGORITHM



i	A[i] <= pivot	Swap(a[i],a[p_index])	++pivot_index
0	A[0]<=17 12<=17 T	Swap(a[0],a[0])	1

- For i=start to end-1
- { if a[i] <= pivot
- { swap(a[i],a[p_index])
- ++pivot_index;
- } }
- Swap(a[end],a[pivot_index])
- Return pivot_index;

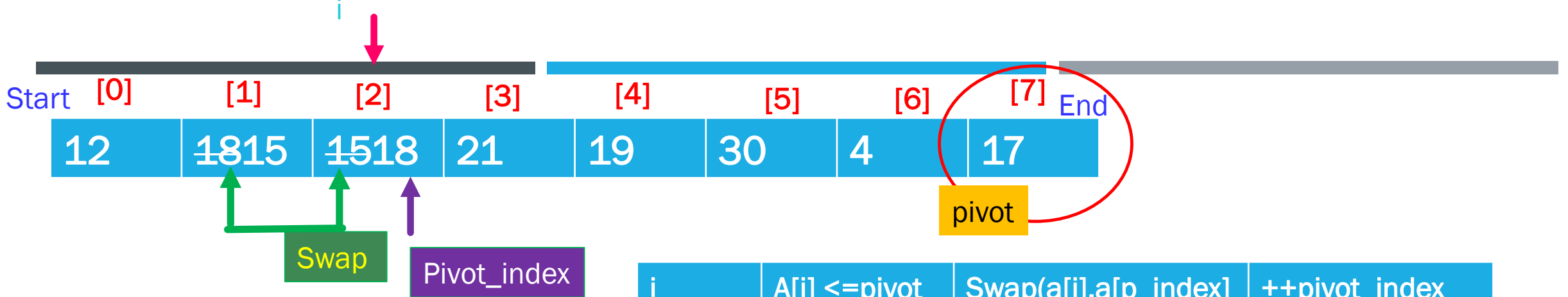
PARTITION ALGORITHM



i	A[i] <= pivot	Swap(a[i],a[p_index])	++pivot_index
0	A[0] <= 17 12 <= 17 T	Swap(a[0],a[0])	1
1	A[1] <= 17 18 <= 17 F	-	-

- For i=start to end-1
- { if a[i] <= pivot
- { swap(a[i],a[p_index])
- ++pivot_index;
- } }
- Swap(a[end],a[pivot_index])
- Return pivot_index;

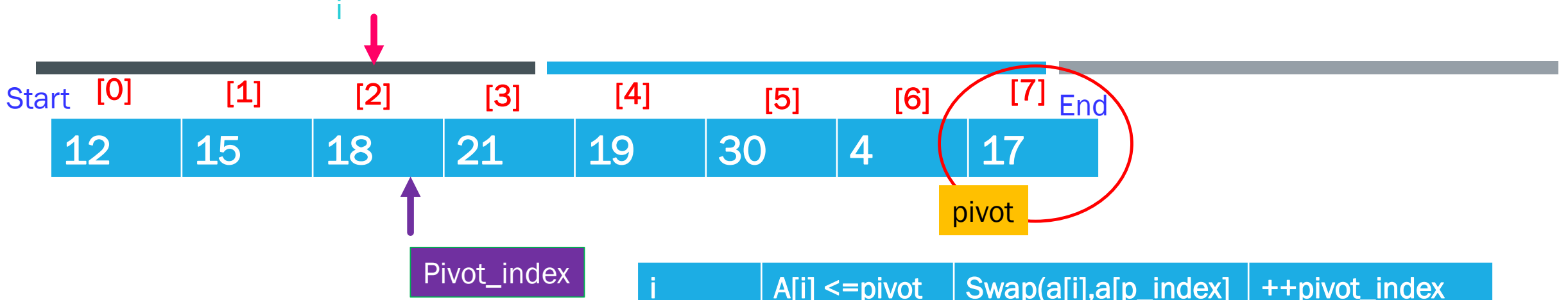
PARTITION ALGORITHM



i	A[i] <= pivot	Swap(a[i],a[p_index])	++pivot_index
0	A[0]<=17 12<=17 T	Swap(a[0],a[0])	1
1	A[1]<=17 18 <= 17 F	-	-
2	A[2]<=17 15 <= 17 T	Swap(a[2],a[1])	2

- For i=start to end-1
- { if a[i] <= pivot
- { swap(a[i],a[p_index])
- ++pivot_index;
- }
- }
- Swap(a[end],a[pivot_index])
- Return pivot_index;

PARTITION ALGORITHM

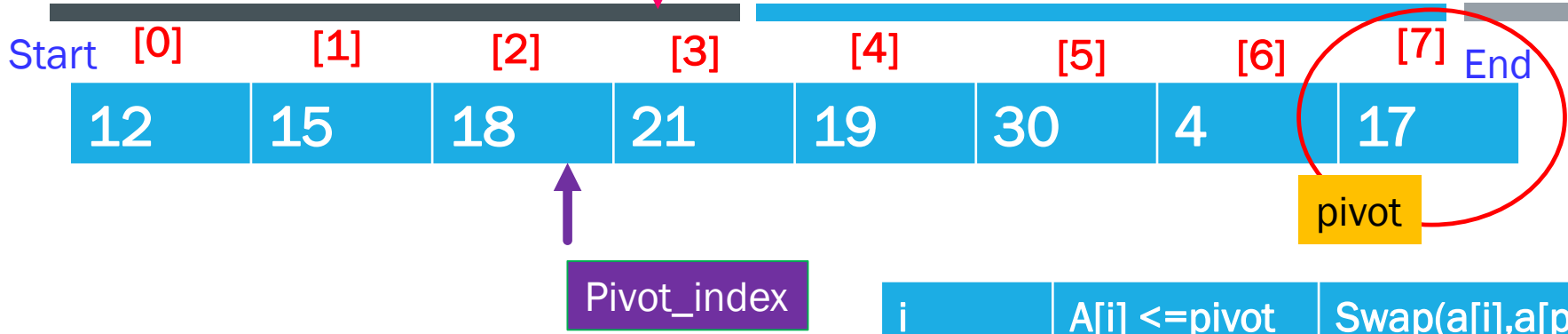


i	A[i] <= pivot	Swap(a[i],a[p_index])	++pivot_index
0	A[0]<=17 12<=17 T	Swap(a[0],a[0])	1
1	A[1]<=17 18 <= 17 F	-	-
2	A[2]<=17 15 <= 17 T	Swap(a[2],a[1])	2

- For i=start to end-1
- { if a[i] <= pivot
- { swap(a[i],a[p_index])
- ++pivot_index;
- } }
- Swap(a[end],a[pivot_index])
- Return pivot_index;

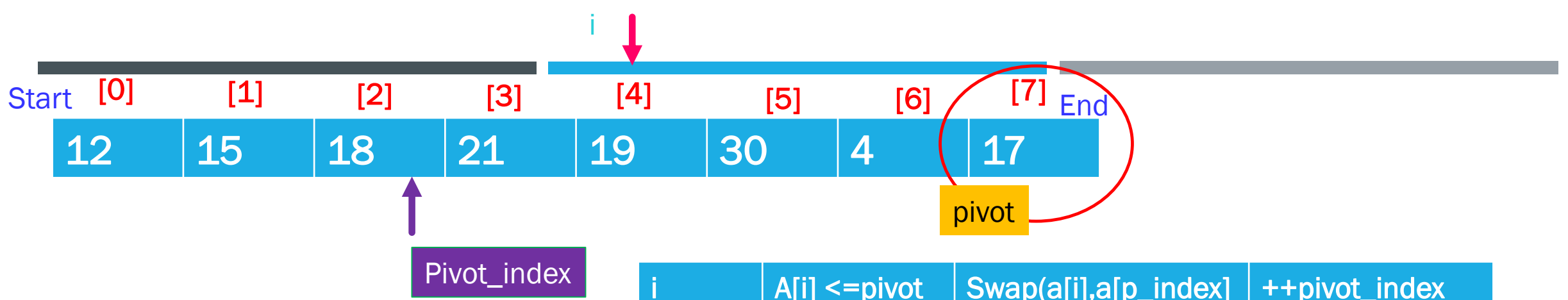
PARTITION ALGORITHM

PARTITION ALGORITHM



- For i=start to end-1
- { if a[i] <= pivot
- { swap(a[i],a[p_index])
- ++pivot_index;
- } }
- Swap(a[end],a[pivot_index])
- Return pivot_index;

i	A[i] <= pivot	Swap(a[i],a[p_index])	++pivot_index
0	A[0]<=17 12<=17 T	Swap(a[0],a[0])	1
1	A[1]<=17 18 <= 17 F	-	-
2	A[2]<=17 15 <= 17 T	Swap(a[2],a[1])	2
3	A[3] <=17 21 <=17 F	-	-

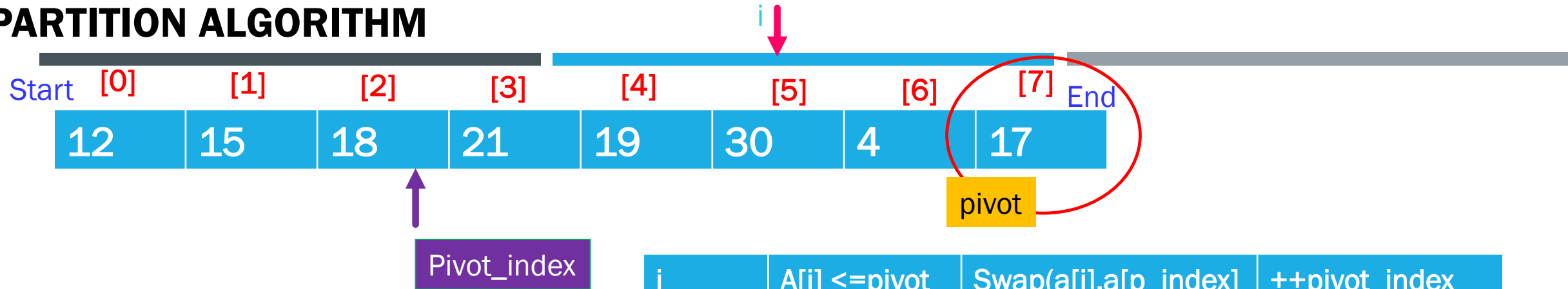


- For i=start to end-1
- { if a[i] <= pivot
- { swap(a[i],a[p_index])
- ++pivot_index;
- } }
- Swap(a[end],a[pivot_index])
- Return pivot_index;

i	A[i] <= pivot	Swap(a[i],a[p_index])	++pivot_index
0	A[0]<=17 12<=17 T	Swap(a[0],a[0])	1
1	A[1]<=17 18 <= 17 F	-	-
2	A[2]<=17 15 <= 17 T	Swap(a[2],a[1])	2
3	A[3] <=17 21 <=17 F	-	-
4	A[4] <= 17 19 <= 17 F	-	-

PARTITION ALGORITHM

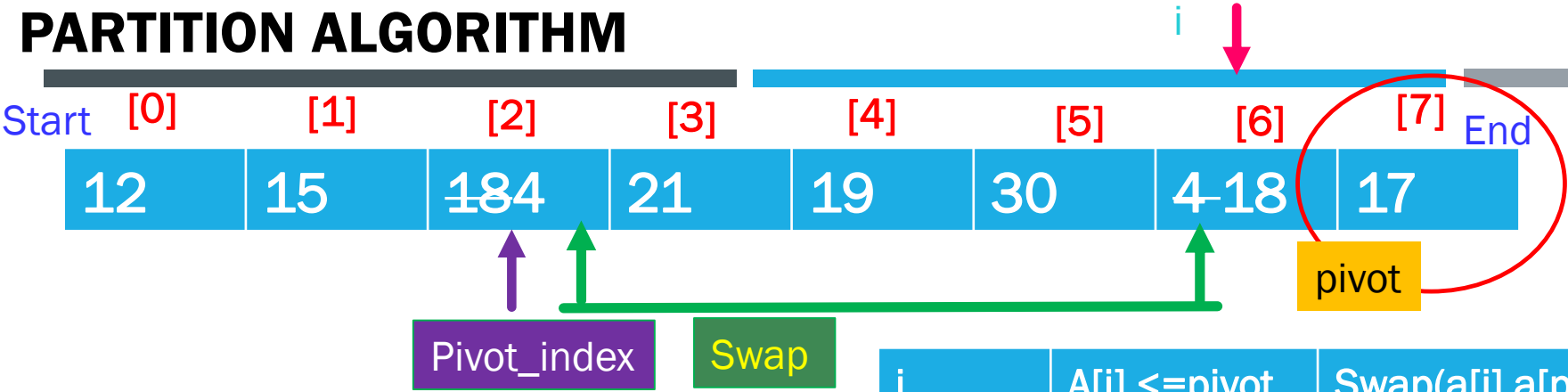
PARTITION ALGORITHM



- For `i=start` to `end-1`
- { if `a[i] <= pivot`
- { swap(`a[i]`,`a[p_index]`)
- ++`pivot_index`;
- } }
- Swap(`a[end]`,`a[pivot_index]`)
- Return `pivot_index`;

i	A[i] <= pivot	Swap(a[i],a[p_index])	++pivot_index
0	A[0]<=17 12<=17 T	Swap(a[0],a[0])	1
1	A[1]<=17 18 <= 17 F	-	-
2	A[2]<=17 15 <= 17 T	Swap(a[2],a[1])	2
3	A[3] <=17 21 <=17 F	-	-
4	A[4] <= 17 19 <= 17 F	-	-
5	A[5] <= 17 30 <= 17 F	-	-

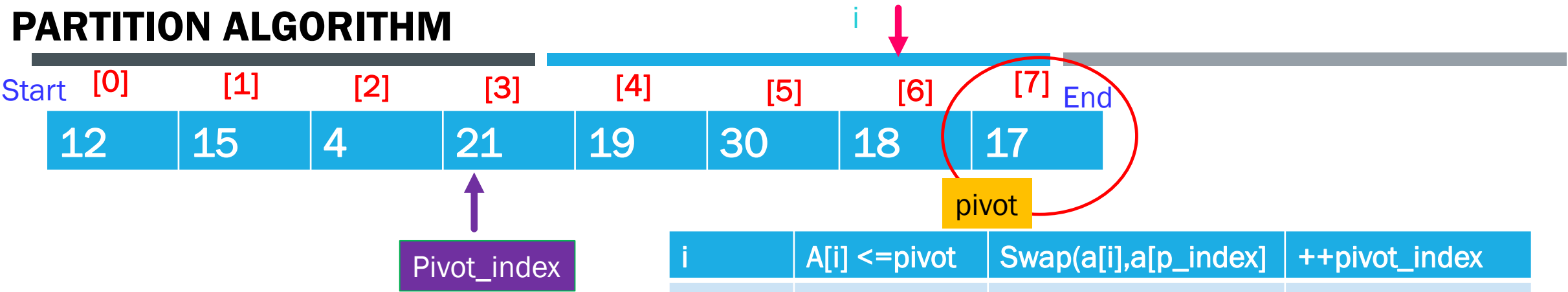
PARTITION ALGORITHM



- For i=start to end-1
- { if a[i] <= pivot
- { swap(a[i],a[p_index])
- ++pivot_index;
- }
- } }
- Swap(a[end],a[pivot_index])
- Return pivot_index;

i	A[i] <= pivot	Swap(a[i],a[p_index])	++pivot_index
0	A[0]<=17 12<=17 T	Swap(a[0],a[0])	1
1	A[1]<=17 18 <= 17 F	-	-
2	A[2]<=17 15 <= 17 T	Swap(a[2],a[1])	2
3	A[3] <=17 21 <=17 F	-	-
4	A[4] <= 17 19 <= 17 F	-	-
5	A[5] <= 17 30 <= 17 F	-	-
6	A[6] <=17 4 <= 17 T	Swap(a[6],a[2])	

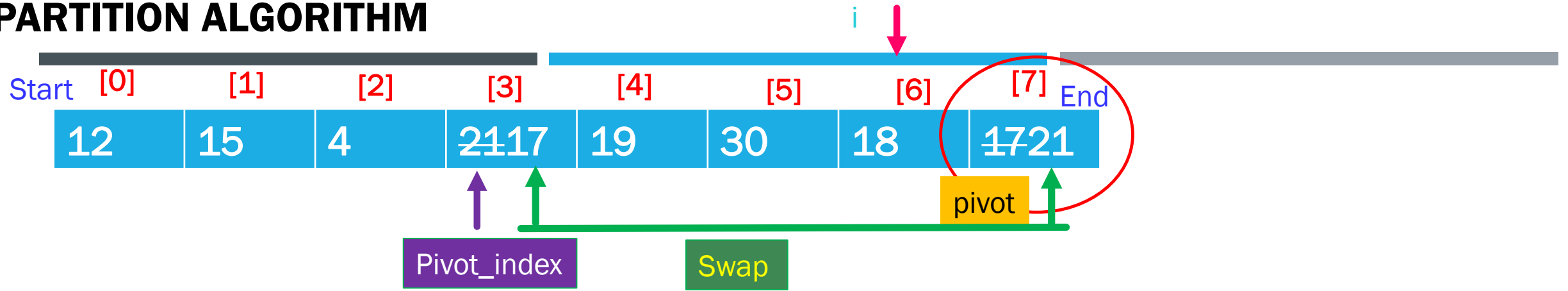
PARTITION ALGORITHM



- For i=start to end-1
- { if a[i] <= pivot
- { swap(a[i],a[p_index])
- ++pivot_index;
- } }
- Swap(a[end],a[pivot_index])
- Return pivot_index;

i	A[i] <= pivot	Swap(a[i],a[p_index])	++pivot_index
0	A[0]<=17 12<=17 T	Swap(a[0],a[0])	1
1	A[1]<=17 18 <= 17 F	-	-
2	A[2]<=17 15 <= 17 T	Swap(a[2],a[1])	2
3	A[3] <=17 21 <=17 F	-	-
4	A[4] <= 17 19 <= 17 F	-	-
5	A[5] <= 17 30 <= 17 F	-	-
6	A[6] <=17 4 <= 17 T	Swap(a[6],a[2])	3
7stop			

PARTITION ALGORITHM

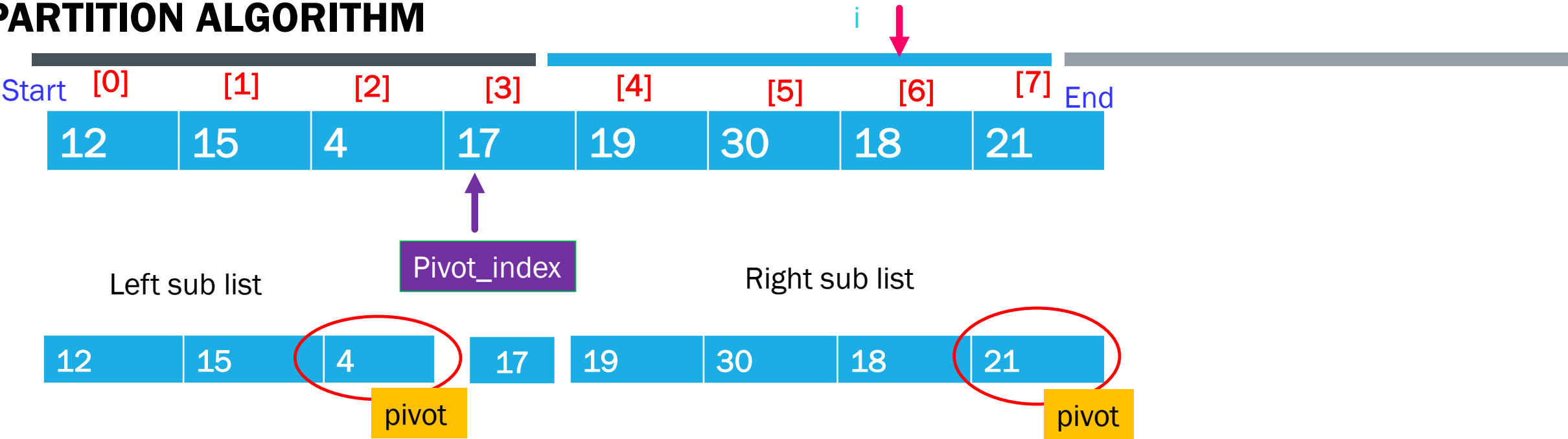


- For i=start to end-1
- { if $a[i] \leq \text{pivot}$
- { swap($a[i], a[\text{p_index}]$)
- ++pivot_index;
- } }
- Swap($a[\text{end}], a[\text{pivot_index}]$)
- Return pivot_index;

Swap($a[\text{end}], a[\text{p_index}]$)

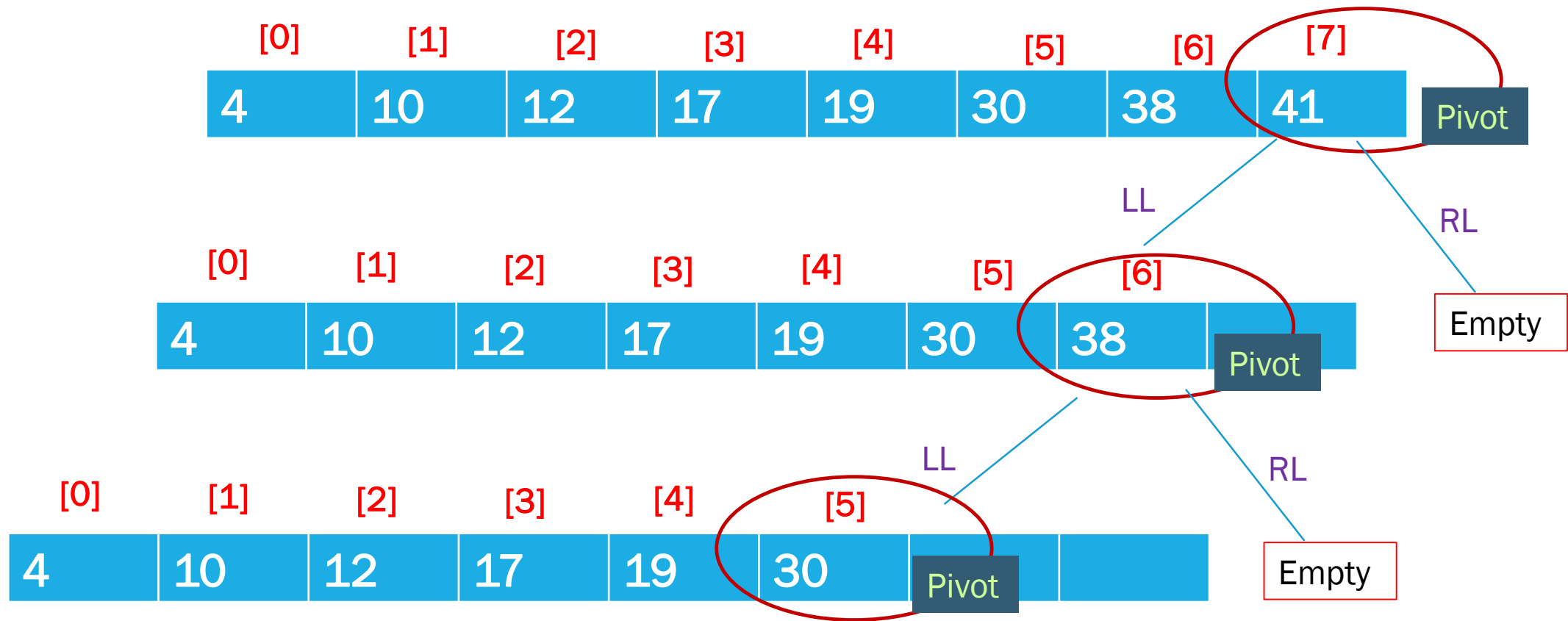
Swap($a[7], a[3]$)

PARTITION ALGORITHM



The same process should be repeated for left sub list and right sub list.

WORST CASE TIME COMPLEXITY OF QUICK SORT



Worst case time complexity of quick sort: $O(n^2)$

RANDOMIZED QUICK SORT

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
12	18	15	21	19	30	4	17

Pivot

- Partition(A, start, end);
- P_index=random(start,end)
- Swap(a[end],a[p_index])

■ For i=start to end-1

■ if $a[i] \leq \text{pivot}$

■ {

■ swap(a[i],a[p_index])

■ ++pivot_index;

■ }

■ Swap(a[end],a[pivot_index])

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
12	18	15	21	19	17	4	30

Pivot



THE END